

Program 1: AI Chatbot for Student Queries (Engineering College)

```
from transformers import pipeline, set_seed

# Load pre-trained language model

chatbot = pipeline("text-generation", model="gpt2")

set_seed(42)

def college_chatbot(query):

    prompt = f"Student Query: {query}\nCollege Bot Answer:"

    response = chatbot(prompt, max_length=120, num_return_sequences=1,
                        pad_token_id=50256)

    return response[0]['generated_text'].split("College Bot Answer:")[-1].strip()

# Knowledge base for common queries (hybrid approach)

faq = {

    "admission": "Admissions are open from June to July. Apply online at the college portal.",

    "fees": "Tuition fee is ₹85,000/year. Scholarships available for merit students.",

    "hostel": "Hostel fees are ₹60,000/year including mess charges.",

    "placement": "Our placement rate is 92%. Top recruiters: TCS, Infosys, Wipro.",

    "timing": "College hours are 9:00 AM to 4:30 PM, Monday to Friday."

}

def answer(query):

    for key in faq:

        if key in query.lower():

            return faq[key]

    return college_chatbot(query)
```

```

print("=== Engineering College Chatbot (type 'exit' to quit) ===")

while True:

    q = input("\nYou: ")

    if q.lower() == "exit":

        break

    print("Bot:", answer(q))

```

Sample Output:

You: What are the admission dates?

Bot: Admissions are open from June to July. Apply online at the college portal.

Program 2: Engineering-Support Chatbot using NLP

```

import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

nltk.download('punkt'); nltk.download('stopwords')

knowledge_base = [

    "To fix a memory leak in Java, close unused objects, avoid static references, and use try-with-resources.",

    "For network latency issues, check ping, use traceroute, and reduce number of hops.",

    "To resolve overheating in a machine, clean the cooling fan and apply thermal paste.",

    "Use version control like Git to manage code changes and collaborate with teams.",

    "For database performance, create indexes on frequently queried columns and normalize tables."

]

```

```
def preprocess(text):
    tokens = word_tokenize(text.lower())
    stop = set(stopwords.words('english'))
    return " ".join(t for t in tokens if t.isalnum() and t not in stop)
```

```
processed_kb = [preprocess(doc) for doc in knowledge_base]
vectorizer = TfidfVectorizer()
kb_vectors = vectorizer.fit_transform(processed_kb)
```

```
def get_solution(question):
    q_vec = vectorizer.transform([preprocess(question)])
    similarity = cosine_similarity(q_vec, kb_vectors)
    idx = similarity.argmax()
    if similarity[0][idx] < 0.1:
        return "Sorry, I couldn't find a relevant solution."
    return knowledge_base[idx]
```

```
print("=== Engineering Support Chatbot ===")
while True:
    q = input("\nYour technical question (exit to quit): ")
    if q.lower() == "exit":
        break
    print("Solution:", get_solution(q))
```

Sample Output:

Question: How do I fix memory leak in Java?

Solution: To fix a memory leak in Java, close unused objects, avoid static references, and use try-with-resources.

Program 3: Text-to-Image Generation (Bridge / Robot)

```
# pip install diffusers transformers torch accelerate

import torch

from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16)

pipe = pipe.to("cuda") # use "cpu" if GPU unavailable

prompt = "A modern suspension bridge over a river at sunset, highly detailed, engineering blueprint style"

image = pipe(prompt, num_inference_steps=30, guidance_scale=7.5).images[0]

image.save("bridge.png")

print("Image generated and saved as bridge.png")
```

Sample Output:

Image generated and saved as bridge.png

Program 4: Compare Images from Different Prompts

```
import torch

from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16).to("cuda")

prompts = [
    "A robotic arm in a factory, realistic photo",
    "A robotic arm in a factory, cartoon style",
```

```
"A robotic arm in a factory, futuristic sci-fi style, neon lights"  
]
```

```
for i, prompt in enumerate(prompts, 1):  
    img = pipe(prompt, num_inference_steps=30).images[0]  
    img.save(f'robot_v{i}.png')  
    print(f'Saved robot_v{i}.png for prompt: {prompt}')
```

Program 5: Speech-to-Text (Engineering Query)

```
# pip install openai-whisper (or use transformers)  
  
import whisper  
  
model = whisper.load_model("base")  
result = model.transcribe("query.wav") # recorded audio file  
  
print("Spoken Query (Transcribed Text):")  
print(result["text"])
```

Alternative using transformers:

```
from transformers import pipeline  
  
asr = pipeline("automatic-speech-recognition", model="openai/whisper-base")  
text = asr("query.wav")["text"]  
print("Transcription:", text)
```

Sample Output:

Program 6: Text-to-Speech (Engineering Text)

```
# pip install gtts  
  
from gtts import gTTS  
  
import os
```

```
text = ("A transformer works on the principle of mutual induction. "
```

```
    "It transfers electrical energy between two circuits "
```

```
    "through a shared magnetic field.")
```

```
tts = gTTS(text=text, lang='en')
```

```
tts.save("engineering_text.mp3")
```

```
os.system("start engineering_text.mp3") # Windows; use 'afplay' on Mac
```

```
print("Speech generated successfully!")
```

Alternative (neural TTS):

```
from transformers import pipeline
```

```
tts = pipeline("text-to-speech", model="facebook/mms-tts-eng")
```

```
speech = tts("Ohm's law states that voltage equals current times resistance.")
```

Program 7: Document Summarization

```
from transformers import pipeline
```

```
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
```

```
document = """
```

Machine learning is a branch of artificial intelligence that enables systems to learn from data without explicit programming. Supervised learning uses labeled datasets to train models for classification and regression tasks.

Unsupervised learning finds hidden patterns in unlabeled data, such as clustering customer segments. Deep learning, a subset of machine learning, uses neural networks with multiple layers to process complex data like images, audio, and text. Applications include autonomous vehicles, medical diagnosis, fraud detection, and natural language processing. Despite its

success, machine learning faces challenges such as data quality, model interpretability, and computational cost.

"""

```
summary = summarizer(document, max_length=60, min_length=25, do_sample=False)
print("Summary:\n", summary[0]['summary_text'])
```

Sample Output:

Summary: Machine learning enables systems to learn from data without explicit programming. It includes supervised, unsupervised, and deep learning, with applications in autonomous vehicles, medical diagnosis, and fraud detection.

Program 8: Machine Translation (English → Hindi)

```
from transformers import pipeline
```

```
translator = pipeline("translation", model="Helsinki-NLP/opus-mt-en-hi")
```

```
english_text = ("Engineering is the application of science and mathematics "
               "to solve real-world problems. Engineers design bridges, "
               "machines, and software systems.")
```

```
result = translator(english_text)
print("Original (English):", english_text)
print("Translated (Hindi):", result[0]['translation_text'])
```

Sample Output:

Translated (Hindi): इंजीनियरिंग वास्तविक दुनिया की समस्याओं को हल करने के लिए विज्ञान और गणित का अनुप्रयोग है।

(For other Indian languages, use appropriate models: opus-mt-en-ta, opus-mt-en-te, etc.)

Program 9: AI Resume Screening & Ranking

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
job_description = """
```

```
Required: Python, Machine Learning, Deep Learning, SQL, Data Analysis,
```

```
NLP, communication skills. Experience with AI model deployment preferred.
```

```
"""
```

```
resumes = {
```

```
    "Candidate_A": "Skilled in Python, machine learning, NLP, SQL. Built AI models and deployed  
them. 3 years experience.",
```

```
    "Candidate_B": "Expert in Java, web development, HTML, CSS. Frontend developer, 5 years  
experience.",
```

```
    "Candidate_C": "Data analyst with Python, SQL, deep learning, and data visualization skills. Strong  
communication.",
```

```
    "Candidate_D": "Mechanical engineer, CAD design, thermodynamics, manufacturing processes."  
}
```

```
docs = [job_description] + list(resumes.values())
```

```
vectorizer = TfidfVectorizer(stop_words='english')
```

```
tfidf = vectorizer.fit_transform(docs)
```

```
jd_vec = tfidf[0]
```

```
scores = cosine_similarity(jd_vec, tfidf[1:]).flatten()
```

```
print("=== Resume Ranking (Job: AI/ML Engineer) ===")
```



```

ranking = sorted(zip(resumes.keys(), scores), key=lambda x: -x[1])

for rank, (name, score) in enumerate(ranking, 1):

    print(f'{rank}. {name} — Match Score: {score:.2f}')

```

Sample Output:

```

=== Resume Ranking (Job: AI/ML Engineer) ===

```

1. Candidate_C — Match Score: 0.58
2. Candidate_A — Match Score: 0.55
3. Candidate_B — Match Score: 0.05
4. Candidate_D — Match Score: 0.00

Program 10: Research Assistance Application

```

from transformers import pipeline

```

```

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

```

```

generator = pipeline("text-generation", model="gpt2")

```

```

def research_assistant(topic):

```

```

    # Generate relevant information

```

```

    info = generator(f'Research on {topic}:', max_length=150,

```

```

                    num_return_sequences=1, pad_token_id=50256)[0]['generated_text']

```

```

    # Generate keywords

```

```

    keywords = [w for w in topic.lower().split() if len(w) > 3]

```

```

    keywords += ["deep learning", "applications", "algorithms", "survey"]

```

```

    # Concise summary

```

```

    summary = summarizer(info, max_length=50, min_length=20)[0]['summary_text']

```

```
print(f"Topic: {topic}")  
  
print(f"\nKeywords: {' '.join(set(keywords))}")  
  
print(f"\nGenerated Information:\n{info}")  
  
print(f"\nSummary:\n{summary}")
```

```
research_assistant("Renewable Energy Systems")
```

Sample Output:

Topic: Renewable Energy Systems

Keywords: renewable, energy, systems, deep learning, applications, algorithms

Summary: Renewable energy systems harness solar, wind, and hydro resources to generate sustainable power and reduce carbon emissions.